Genome Informatics 2026

Lesson 3

Lesson overview

- AI Agents in bioinformatics
- Cancer analysis
- We use git

AI Agents in bioinformatics

Part I - Foundations

1. What are AI agents?
2. Why bioinformatics needs agents
3. Introduction to Claude Code
4. Architecture & core principles

Part II - Commands & Planning

1. Essential Claude commands
2. The /plan workflow
3. Memory & context management
4. Tool integration (MCP)

Part III - Examples

1. Example 1 · FASTQ QC script
2. Example 2 · BWA-MEM2 alignment
3. Example 3 · Variant calling pipeline

What Are AI Agents?

Autonomous systems that perceive, reason, and act

- Software entity that can perceive, reason, act and learn to achieve a goal
- Decides what to do next rather than just responding to a single prompt
- Definition: An entity that observes its environment through sensors and acts upon that environment through actuators to maximize a reward or accomplish a task
- “Environment” is often:
  - files
  - APIs
  - web pages
  - databases
  - user instructions

What Are AI Agents?

Autonomous systems that perceive, reason, and act

A typical agent architecture consists of:

1. Perception layer
   a. Collects input: user prompt, documents, tool outputs
2. Reasoning / planning
   a. Decides what to do next
   b. Often implemented with an LLM that generates intermediate steps (chain-of-thought, plans, or tool calls)
3. Tools / actions (agent selects)
   a. web search
   b. code execution
   c. database queries
   d. sending emails
4. Memory
   a. Short-term: conversation context
   b. Long-term: stored knowledge, embeddings, or logs
5. Control loop
   a. observe → think → act → observe → …

What Are AI Agents?

Autonomous systems that perceive, reason, and act

Definition

An AI agent is a software system that uses a large language model (LLM) as its reasoning engine to autonomously plan, call tools, and iterate toward a goal - without requiring step-by-step human instructions.

Key properties

- ✦ Perception - reads files, tools, APIs, databases
- ✦ Reasoning - LLM plans multi-step solutions
- ✦ Action - writes code, runs commands, edits files
- ✦ Memory - retains context across steps
- ✦ Iteration - self-corrects on errors

The Agent Loop

1 OBSERVE

Read codebase, files, error output

2 THINK

LLM reasons: what needs to happen?

3 ACT

Call tools: Bash, Edit, Write, Grep…

4 EVALUATE

Check result, parse errors, re-plan

→ *repeats until task complete*

Why Bioinformatics Needs AI Agents

The field is data-rich, tool-heavy, and code-intensive

Volume & Variety

A single WGS run produces ~100 GB of raw reads.
Dozens of formats: FASTQ, BAM, VCF, GFF, HDF5, AnnData…

Pipeline Complexity

Typical pipelines chain 10–20 tools (bwa, samtools, GATK, DESeq2…) with hundreds of parameters — hard to remember, easy to misconfigure.

Reproducibility Crisis

>70% of published analyses cannot be reproduced.
AI agents can generate documented, version-locked workflows automatically.

Knowledge Breadth

A single project may require statistics, genomics, cloud computing, Python/R/shell scripting, and domain biology simultaneously.

Introduction to Claude Code

Anthropic's agentic CLI for software engineering

What it is

- CLI tool: claude (interactive REPL)
- Powered by Claude Sonnet / Opus 4.x
- Runs locally - reads your actual files
- Full tool access: Bash, Edit, Write, Grep, Glob…
- Persistent memory across sessions

What it is NOT

- Not just a chatbot - it takes real actions
- Not a black box - every tool call is shown
- Not cloud-only - works offline for local ops
- Not a one-shot generator - iterates & fixes errors
- Not language-locked - Python, R, shell, nextflow…

Why it matters for bioinformatics

- Understands genomics domain vocabulary
- Generates + runs pipelines end-to-end
- Reads BAM/VCF/FASTQ and interprets results
- Integrates MCP servers (NCBI, UniProt, etc.)
- Explains code for teaching & collaboration

Claude Code - Core Principles

How to get the best results from an AI coding agent

Be Specific

Give Claude the file paths, tool names, and expected output.
"Align sample.fastq to hg38 using bwa-mem2 and produce a sorted BAM" is far better than "align my reads".

Work Top-Down with a Plan

For multi-step tasks always start with /plan.
Claude will decompose the problem, identify files, and list steps before writing a single line of code.

Iterate, Don't Rephrase

If a step fails, say what went wrong and let Claude fix it.
Avoid restarting - Claude retains context and can self-correct within the same session.

Trust but Verify Tool Calls

Every Bash / Edit / Write call is shown before execution.
Review destructive operations (rm, overwrite) before approving.

Use Memory for Recurring Context

Save project conventions, tool versions, and data paths to memory.
Claude loads these automatically in future sessions.

Scope Actions to the Request

Claude should only do what was asked - no unrequested refactoring.
If you want only alignment, say so; don't let the agent expand scope.

Essential Claude Commands — Part 1

Slash commands that shape how Claude works

/plan

Ask Claude to design a step-by-step implementation plan before writing any code. Returns files to edit, decisions, and trade-offs. Use for any task longer than ~3 steps.

```
$ /plan align paired-end FASTQ files with bwa-mem2, produce sorted indexed BAM, compute coverage
```

/help

Show all available commands, tools, and keyboard shortcuts. Also explains how to configure permissions and hooks.

```
$ /help
```

/clear

Reset the conversation context window while keeping the project memory intact. Use when switching to a new sub-task to avoid context pollution.

```
$ /clear
```

/compact

Summarise the current conversation into a compressed form, freeing context space for long-running sessions without losing the essential state.

```
$ /compact
```

/context

Detailed breakdown of how your 200,000-token context window is being used.

```
$ /context
```

Essential Claude Commands — Part 2

Task management, review, and commit workflows

/commit

Stage relevant files and create a well-formed git commit with an auto-generated message. Claude reviews the diff before committing and warns about sensitive files.

```
$ /commit  → stages, generates message, commits
```

/review-pr

Review an open GitHub pull request: reads diff, runs tests, checks for bugs and security issues, posts inline comments.

```
$ /review-pr 42
```

/init

Scan the repository and create CLAUDE.md — a project context file that persists conventions, key files, and tool instructions for every future session.

```
$ /init  → creates CLAUDE.md at repo root
```

/doctor

Diagnose the Claude Code environment: check authentication, permissions, MCP server connections, and hook configuration.

```
$ /doctor
```

/model

Toggle mode (Sonnet → Haiku or Opus → Sonnet). Use for quick edits; switch back for complex reasoning.

```
$ /model  (toggle)
```

Let’s try some prompt!

Write and execute source/aln.py script that performs bwa-mem2 alignment of the sample in @data/dna and visualize these alignments against reference genome

Why Create a Plan Before Coding?

The single most important habit when using Claude Code

Without a plan, Claude will make local decisions that seem reasonable step-by-step but can produce an incoherent whole.
A plan forces alignment between you and the agent before any file is touched.

Catch Misunderstandings Early

The plan phase reveals if Claude misunderstood the goal.
It costs nothing to correct a plan; fixing wrong code costs time.

Reduces Wasted Context

Each Claude response consumes context tokens.
A plan ensures tokens are spent on the right problem.

Enables Human Review

You can approve, modify, or reject individual steps before Claude executes anything destructive.

Forces Decomposition

Complex bioinformatics tasks have hidden dependencies (index before align, sort before index…). Planning surfaces these.

Creates Documentation

The plan text becomes inline documentation for the pipeline, making the code easier to review and teach.

How to Create an Effective Plan

Anatomy of a good /plan prompt and what to look for in the output

Writing the Plan Prompt

1. State the goal in one sentence
2. Name input files & formats
3. Name output files & formats
4. List tools / versions to use
5. State any constraints
6. Ask for alternatives if unsure

What to Look for in the Plan Output

- ✓ **Numbered steps in correct dependency order**
- ✓ **Each step names the tool / function to use**
- ✓ **Critical files identified by path**
- ✓ **Trade-offs explicitly stated**
- ✓ **Risks / assumptions called out**
- ✗ **Steps that skip index-before-use**
- ✗ **Hardcoded paths that differ from your layout**
- ✗ **Missing error handling for tool failures**
- ? Ask Claude to split any step > ~10 lines
- ? Ask for a fallback if a tool is unavailable

Example 1 · FASTQ Quality Check (Simple)

First contact: generate a QC report from raw reads in one prompt

Prompt: "Write source/qc.py that reads data/dna/example_human_Illumina.pe_1.fastq, computes per-read mean Phred quality and GC content, plots histograms, saves to output/qc/qc_report.png"

What Claude does

- Reads FASTQ with plain Python (no extra libs)
- Computes mean Phred per read (ASCII-33)
- Computes GC% per read
- Plots two histograms with matplotlib
- Saves PNG to output/qc/

Concepts demonstrated

- FASTQ 4-line record structure
- Phred quality score encoding
- GC content as genome quality proxy
- Histogram as read-level QC
- Path handling with pathlib

What you learn as a user

- Claude understands bioinformatics file formats
- Code is readable and runnable immediately
- No hallucinated imports only stdlib + matplotlib
- Output directory is created automatically
- Extend easily: add R1 vs R2 comparison

★★★★★ BASIC

Example 2 · BWA-MEM2 Alignment Pipeline

Chaining tools: index → align → sort → index → stats → visualise

Prompt: "/plan Write source/aln.py: bwa-mem2 alignment of data/dna paired-end reads to reference, sorted BAM, flagstat, depth, mpileup base composition, matplotlib visualisation."

bwa-mem2 index	Build FM-index (.bwt.2bit.64, .0123, …)
bwa-mem2 mem	Align paired FASTQ → SAM with @RG tag
samtools sort	SAM → coordinate-sorted BAM
samtools index	BAI random-access index
samtools flagstat	Mapping rate, pairing stats
samtools depth	Per-base coverage depth
samtools mpileup	Base composition per position
matplotlib	8-panel PDF report

★★★★☆ INTERMEDIATE

Example 3 · RNA-seq Differential Expression

Statistical analysis: count normalisation, DE testing, MA + volcano plots

Prompt: "/plan Write source/rna_de.py using pydeseq2. Load data/rna/counts.csv (rows=genes, cols=samples), metadata from data/rna/meta.csv. Run DESeq2 normalisation + Wald test. Output: MA plot, volcano, top-20 heatmap, results CSV."

Plan steps Claude produces

1. Load & validate count matrix dimensions
2. Build DESeqDataSet with design ~ condition
3. Estimate size factors (normalisation)
4. Estimate dispersions
5. Fit negative binomial GLM + Wald test
6. Filter: padj < 0.05, |log2FC| > 1
7. MA plot: log2FC vs mean expression
8. Volcano: -log10(pval) vs log2FC
9. Heatmap: top-20 DE genes (seaborn)
10. Save results/rna/de_results.csv

Key bioinformatics concepts covered

- Raw counts vs normalised expression
- Negative binomial distribution for counts
- Multiple testing correction (BH/FDR)
- MA plot interpretation
- Volcano plot: significance vs effect size
- Heatmap clustering (Ward linkage)

Claude Code advantages here

- Knows pydeseq2 API without docs lookup
- Handles missing samples gracefully

★★★★☆
MODERATE

Example 3 · Variant Calling Pipeline

End-to-end: alignment → mpileup → call → annotate → visualise

Prompt: "/plan Extend source/aln.py with variant calling: bcftools mpileup + call -mv, filter QUAL>20 & DP>10, annotate with snpEff if available, plot SNP/INDEL spectrum, Ti/Tv ratio, QUAL distribution, save VCF + summary PNG."

Sorted BAM → **bcftools mpileup** → **bcftools call -mv** → **bcftools filter** → **snpEff annotate** → **Plots & Report**

bcftools mpileup options

- --no-BAQ (disable BAQ recalibration for speed)
- -d 10000 (max depth cap per position)
- -f REF (reference FASTA for genotyping)
- Produces pileup: chr, pos, ref, depth, bases, quals

bcftools call / filter

- call -mv (multiallelic model, variants-only)
- filter QUAL > 20 removes low-confidence calls
- filter DP > 10 removes low-coverage sites
- Output: PASS-tagged VCF

Visualisation panels

- SNP/INDEL count bar chart
- Substitution spectrum (6 canonical classes)
- Ti/Tv ratio (expected ~2.0–2.1 in WGS)
- QUAL score distribution with PASS threshold

★★★★☆ ADVANCED

Example 5 · Multi-omics Integration Agent (Complex)

Autonomous agent that integrates WGS variants + RNA-seq expression + clinical data

Prompt: "/plan Build source/multiomics_agent.py. Input: VCF (variants), counts.csv (RNA-seq), clinical.csv. Steps: (1) filter coding variants, (2) intersect with DE genes, (3) cluster patients by omics profile, (4) survival analysis, (5) generate report.pdf with all figures."

WGS Layer	RNA-seq Layer	Clinical Layer	Integration	Output
VCF → coding variants (missense, nonsense, frameshift) Gene-level burden score	Count matrix → DESeq2 Differentially expressed genes log2FC + adjusted p-value	Patient metadata Survival time + event Clinical covariates	Variant × expression overlap Multi-omics matrix (MOFA+) Patient clustering (k-means)	Kaplan-Meier curves OncoPrint heatmap report.pdf (10 figures)

Agent sub-tasks Claude plans automatically

- parse_vcf() → filter functional_impact in [HIGH, MODERATE]
- run_deseq2() → returns DataFrame with gene, log2FC, padj
- intersect(vcf_genes, de_genes) → candidate driver genes
- build_mofa_matrix() → samples × features (variants + expression)
- cluster_patients(n_clusters=4) → assigns cluster labels
- kaplan_meier(time, event, cluster) → log-rank test + plot
- oncoprint(top30_genes, samples) → alteration matrix heatmap
- render_pdf(figures) → reportlab multi-page report

★★★★★ EXPERT

Example in Claude code

Create plan ALN.md to create aln.py that performs bwa-mem alignment of the sample in @data/dna and visualize these alignments against reference genome

Best Practices for Claude Code in Bioinformatics

Lessons learned from real pipeline development

Always /plan for multi-step pipelines

One prompt → one plan → approval → execution.
Never let Claude invent a 10-step pipeline without reviewing the plan.

Pin tool versions in CLAUDE.md

Add: "Use bwa-mem2 2.2.1, samtools 1.21, bcftools 1.21."
Claude reads CLAUDE.md at session start — no need to repeat.

Use /commit after each working milestone

Keep git history granular: index, align, flagstat as separate commits.
Easy to bisect if a later step breaks the pipeline.

Approve destructive operations explicitly

Never auto-approve rm, overwrite, or force-push.
Claude will ask — read the diff before confirming.

Use memory for recurring context

Save: reference genome path, sample naming convention, cluster SLURM config.
Claude loads ~/.claude/projects/<repo>/MEMORY.md automatically.

Test on small data first

Use 1000-read subsets for pipeline development.
Ask Claude: "Subset R1/R2 to 1000 reads for testing."

Memory & Context Management

How Claude retains knowledge across sessions

/user - Who you are

Role, expertise level, preferred language.
Example: "I am a computational biologist. I prefer Python over R for data pipelines."

/project - What you're building

Active task, deadlines, design decisions.
Example: "Working on TCGA BRCA multi-omics analysis. Freeze branch before 2026-04-15."

/feedback - How to work with you

Corrections Claude should remember.
Example: "Do not use mock databases in tests - use real SQLite fixture files instead."

/reference - Where things live

External systems: databases, dashboards, repos.
Example: "Reference genome: /data/refs/hg38.fa. Pipeline bugs tracked in Linear project PIPE."

Tool Integration via MCP Servers

Model Context Protocol: connect Claude to any external system

MCP (Model Context Protocol) lets Claude connect to external tools and databases as structured tool calls - the same way it calls Bash or Edit internally. Each MCP server exposes typed tools that Claude discovers and calls autonomously.

NCBI Entrez

Search PubMed, fetch gene records, download sequences from NCBI.

```
fetch_gene(gene_id='BRCA1', db='gene')
```

UniProt

Retrieve protein sequences, functional annotations, domains.

```
get_protein(accession='P38398')
```

Ensembl REST

Gene coordinates, transcript models, variant consequence prediction.

```
vep(chrom='17', pos=41276045, ref='G', alt='A')
```

Galaxy / Nextflow

Submit & monitor workflow runs on HPC or cloud clusters.

```
submit_workflow(wf='rnaseq', inputs={...})
```

SQLite / PostgreSQL

Query local or remote databases for sample metadata & results.

```
query('SELECT * FROM samples WHERE cohort=?', ['TCGA'])
```

Filesystem MCP

Extended file operations: zip, checksum, remote sync (rsync/S3).

```
upload_s3(path='output/', bucket='my-bucket')
```

Reproducibility & Documentation

The underrated superpower of AI-assisted bioinformatics

Science requires reproducibility. Claude Code can generate reproducible pipelines by default — if you ask for it.
Key: treat documentation as a first-class deliverable, not an afterthought.

CLAUDE.md — Project Context File

Created by /init. Contains: tool versions, input/output paths, pipeline conventions, data sources. Loaded automatically every session. Acts as a living README for both humans and Claude.

Locked Dependencies

Ask Claude: "Add requirements.txt with pinned versions for all imports." For conda: "Generate environment.yml with exact versions." For containers: "Write Dockerfile for this pipeline."

Automated Git History

Use /commit after each step. Claude writes semantic commit messages: "feat: add bwa-mem2 alignment step with @RG tag" — not "update script". The full analysis history is in git, not in your head.

Self-Documenting Code

Ask Claude: "Add docstrings to every function explaining the bioinformatics rationale, not just the code logic." Result: code that teaches the next researcher reading it.

Snakemake / Nextflow Translation

Once a prototype works, ask: "Convert source/aln.py to a Snakemake workflow with wildcard samples." Claude generates the Snakefile, config.yaml, and cluster profile.

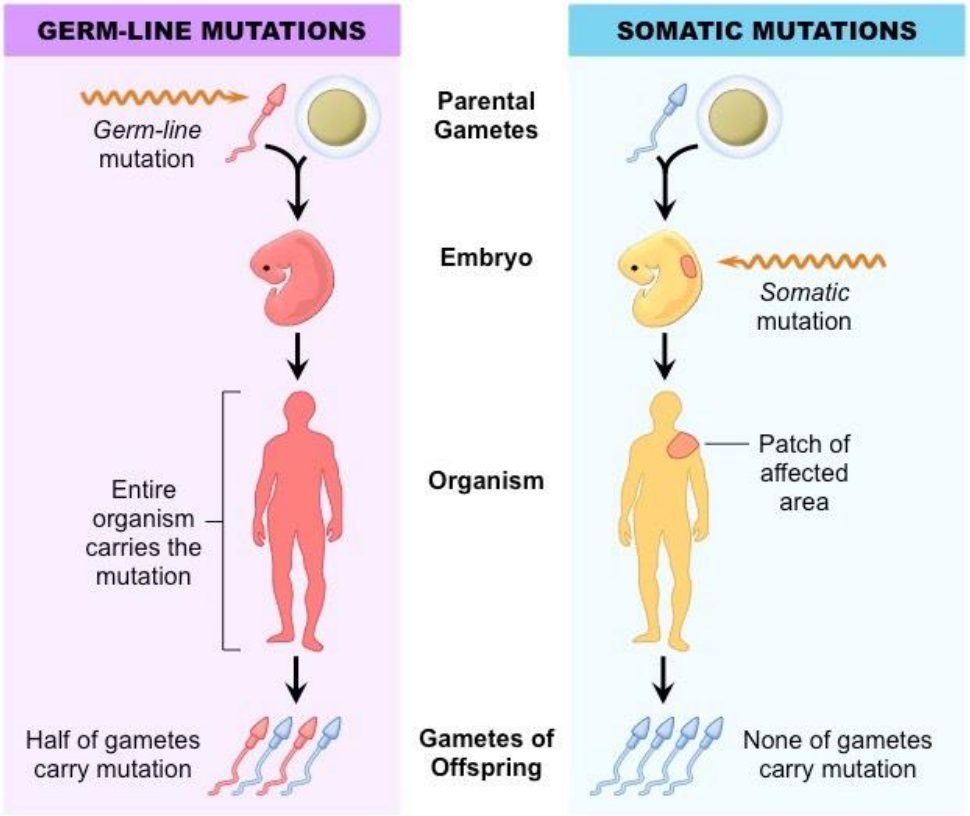
Key Takeaways

1. AI agents don't just generate code — they plan, act, and self-correct.
2. Always /plan before multi-step bioinformatics pipelines.
3. Claude Code knows FASTQ, BAM, VCF, and tool CLIs natively.
4. Memory + CLAUDE.md eliminate repetitive context setup.
5. MCP servers extend Claude to NCBI, UniProt, HPC schedulers.
6. Reproducibility is a prompt away: pinned deps, Snakemake, Dockerfiles.
7. Start simple (FASTQ QC) → build to complex (multi-omics agent).

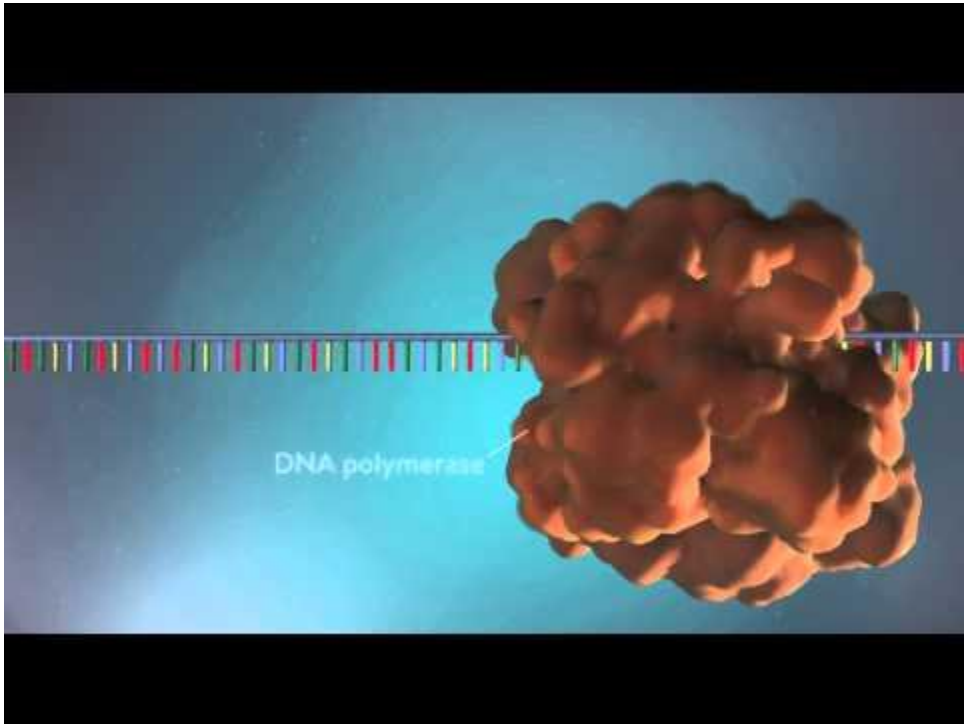
Thank you · Questions?

Computational Cancer Analysis

Variants?


GERM-LINE MUTATIONS
SOMATIC MUTATIONS
Parental Gametes
Germ-line mutation
Embryo
Somatic mutation
Organism
Entire organism carries the mutation
Patch of affected area
Gametes of Offspring
Half of gametes carry mutation
None of gametes carry mutation

DNA replication



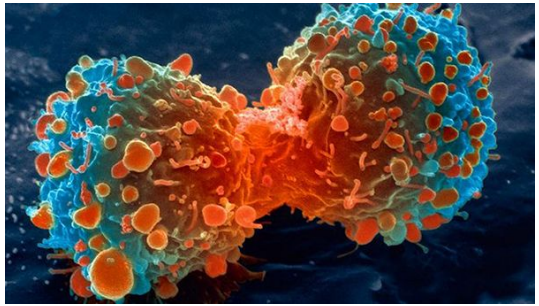
What is cancer?

Mutation during DNA replication can fall to:

1. Intron (no change)
2. Important gene (cell dies, organism lives)
3. Gene that stops cell division (cell lives, organism...)

What causes cancer (increases probability of mutation)?

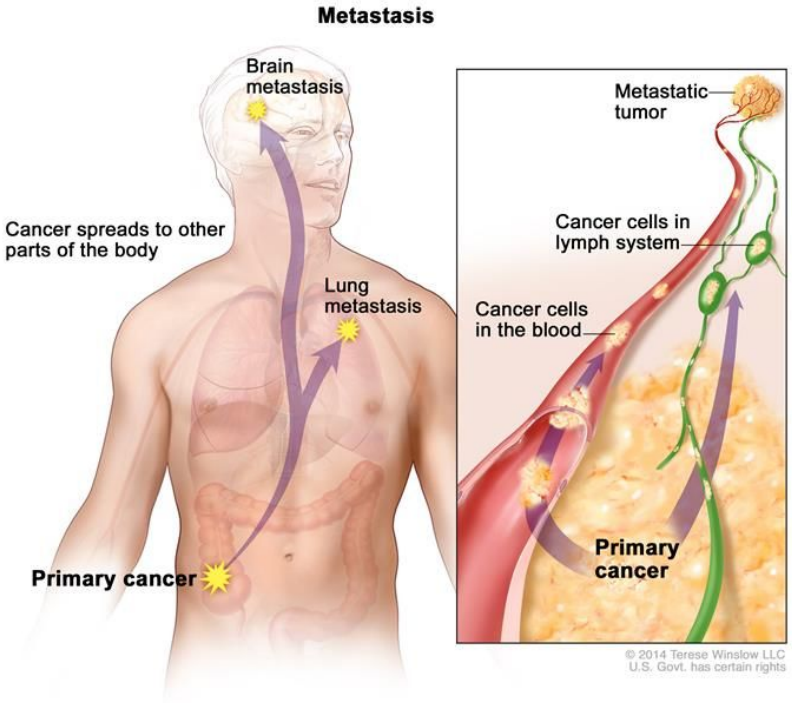
1. EM radiation
2. Chemical agents
3. Free radicals
4. Genetic factors
5. Infections (viruses)



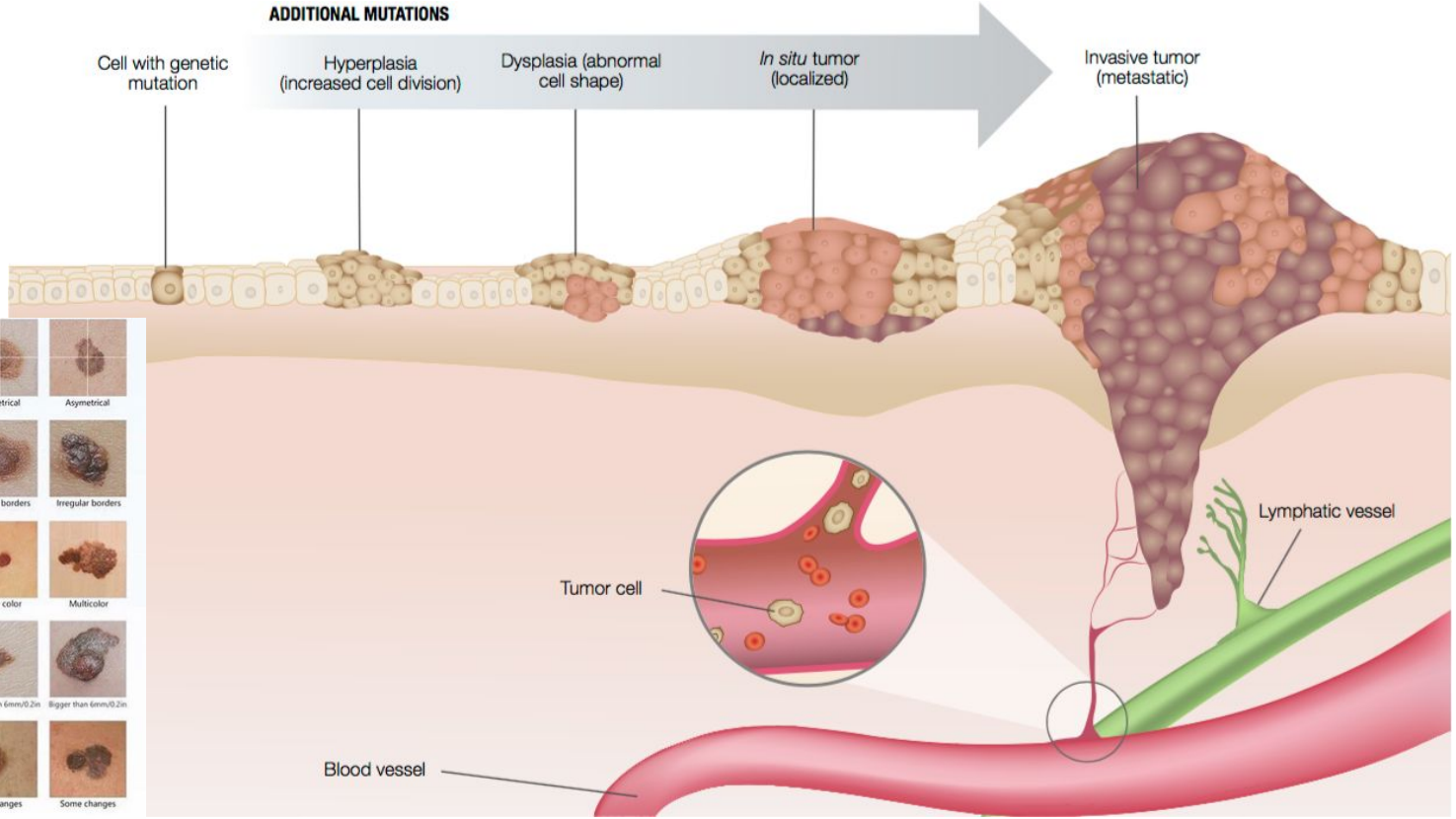
A dividing lung cancer cell.
Credit: National Institutes of Health

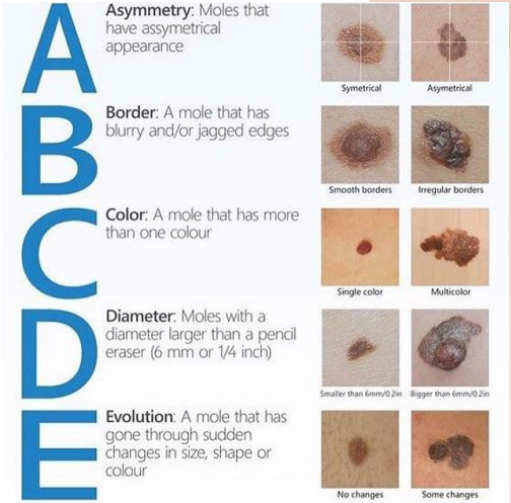
What is metastasis?

Body's cells begin to divide without stopping and spread into surrounding tissues
Cancer cells - ignore signals that normally tell cells to stop dividing or that begin a process known as programmed cell death, or **apoptosis**, which the body uses to get rid of unneeded cells



Increased epitel cell division leading to metastasis





"Drivers" of Cancer

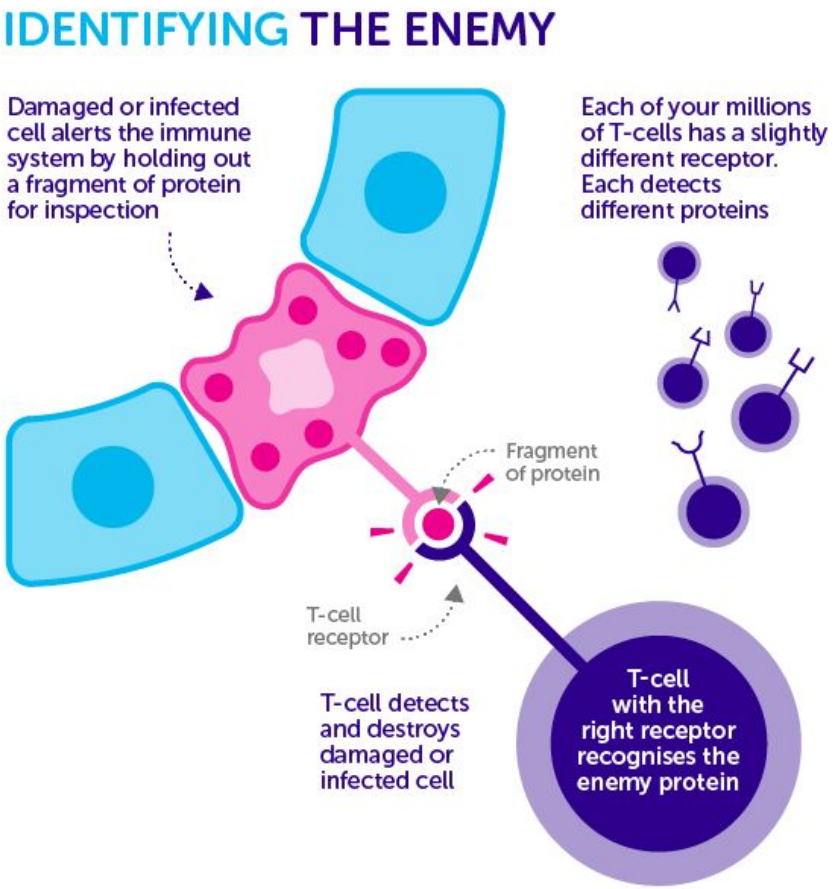
Cancer is a genetic disease that is caused by changes to genes which control the way our cells function, especially how they grow and divide:

1. **Abnormal growth (proto-oncogenes)** Cellular growth mechanism is damaged and cell starts to multiply uncontrollably
2. **Damaged control mechanism** (tumor suppressors) - Cells with certain alterations in tumor suppressor genes may divide in an uncontrolled manner (TP53 - Apoptosis)
3. **Damaged DNA repair mechanism** (Accumulated errors in this group of genes can lead to uncontrollable proliferation)

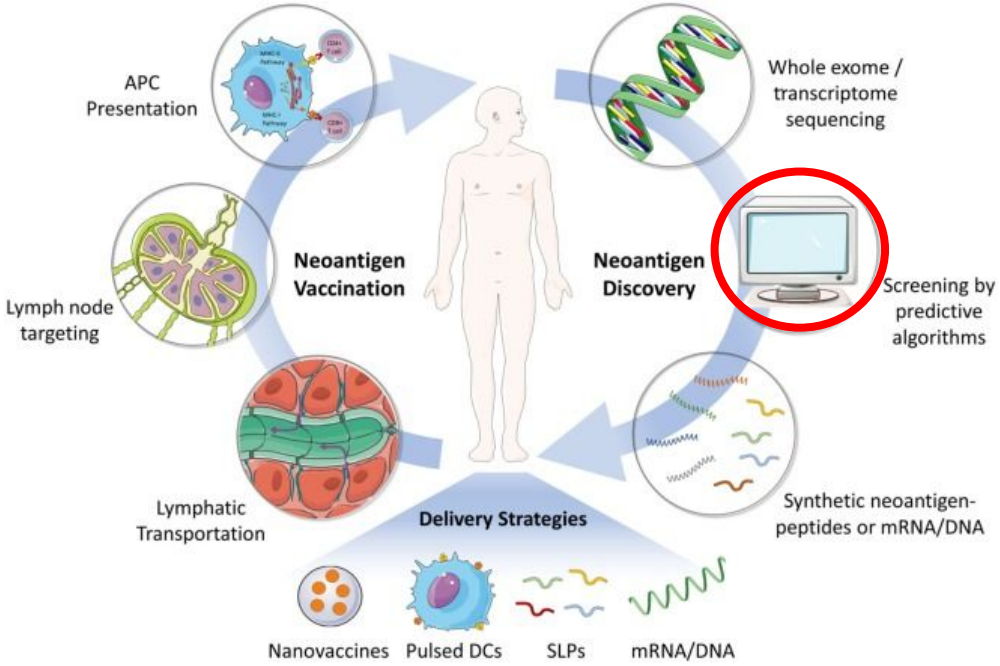
Cancer cells

Our body develops thousands cancer cells every day.

OMG! OMG! OMG!



Neoantigens

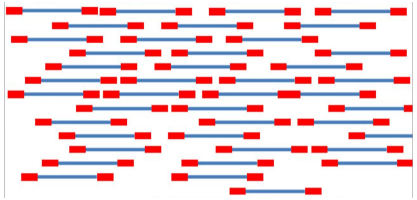


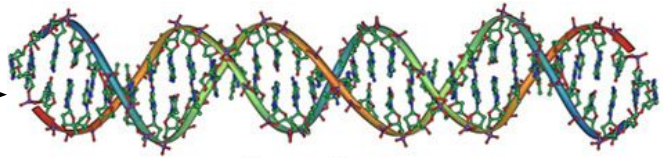
- Neoantigens - proteins presented only by cancer cells
- When neoantigen is known -> immune T-cells can be “programmed” to destroy cancer cells
- These unique cancer markers could be a key to developing a new generation of personalized, targeted cancer immunotherapies

Yugang Guo et all. Neoantigen Vaccine Delivery for Personalized Anticancer Immunotherapy

How can we discover neoantigens?

1. Reconstruct DNA of tumor and normal tissue

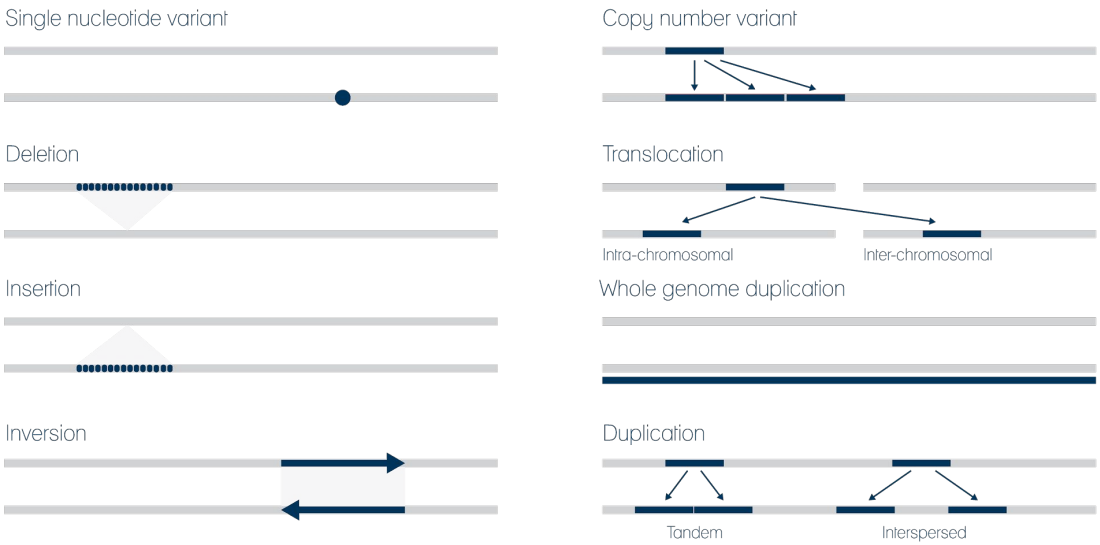




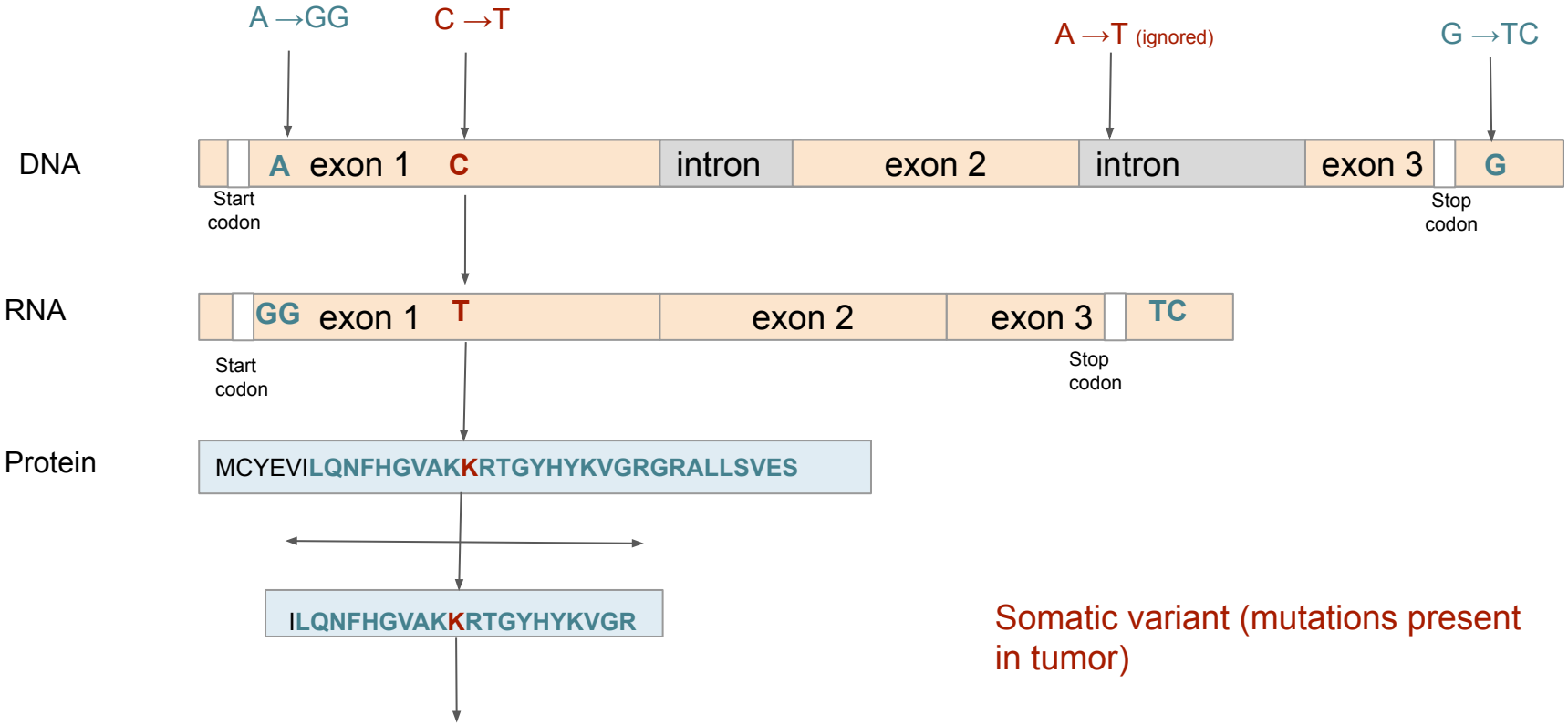
Genome Sequence

How can we discover neoantigens?

2. Compare DNA from Tumor and Normal tissue



How can we discover neoantigens?



Somatic variant (mutations present in tumor)

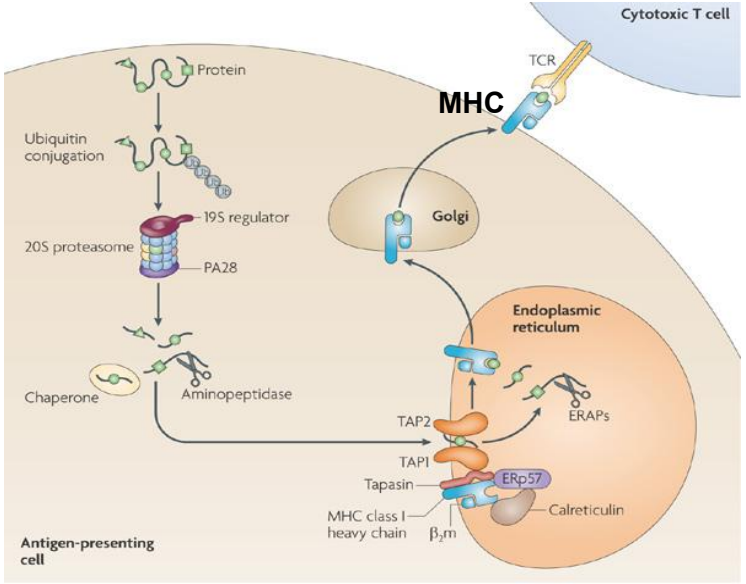
3. Protein extraction

How can we discover neoantigens?

4. Discover HLA type from genome (translates to MHC molecule)

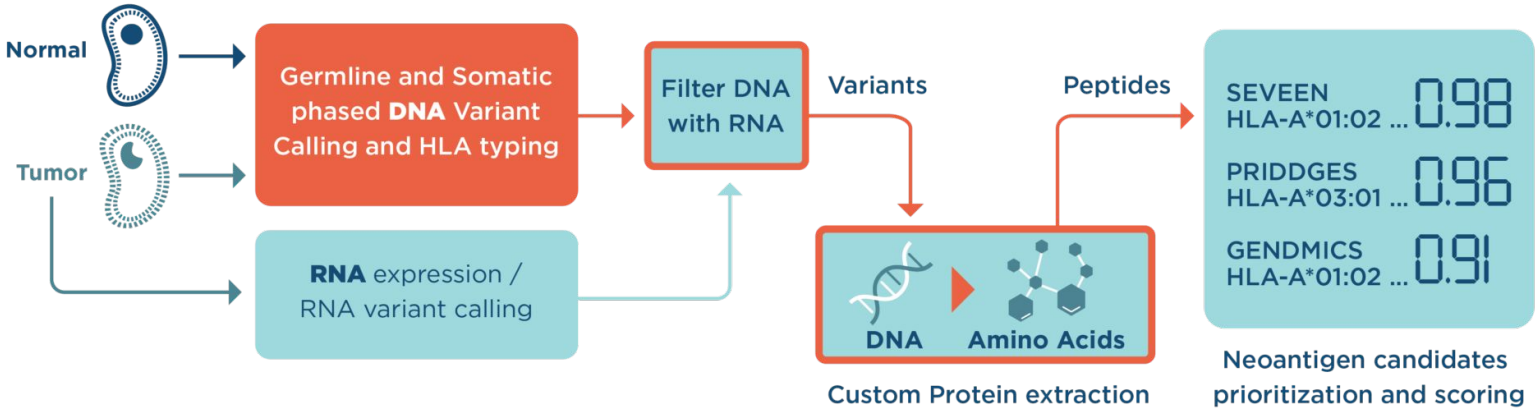
5. Perform scoring of protein-HLA sets (IEDB processing tools)

Mutation	HLA type	peptide	NetMHC score	Pickpocket score	NetCTLPan score	RNA expression
1_111957245_C_A	HLA-A*02:01	MMLSSSPV	0.881	0.633	1.09815	11.5
8_144392368_T_C	HLA-A*02:01	WLLEKLEQL	0.828	1.097	1.06133	12.5
17_28537638_C_T	HLA-A*02:01	VLDEFPHV	0.836	0.374	1.06015	23

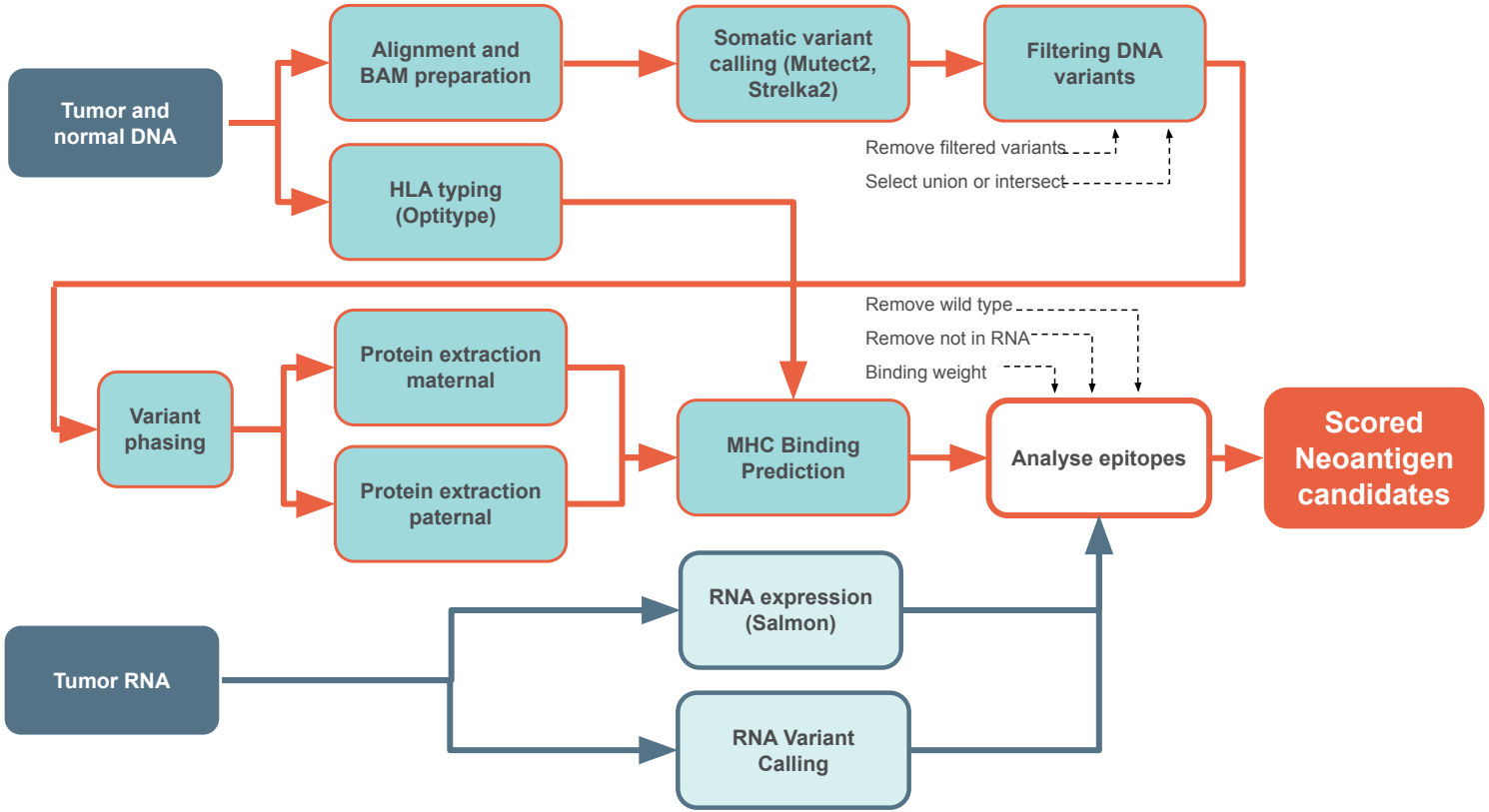


Nature Reviews | Immunology

Neoantigen Discovery Workflow



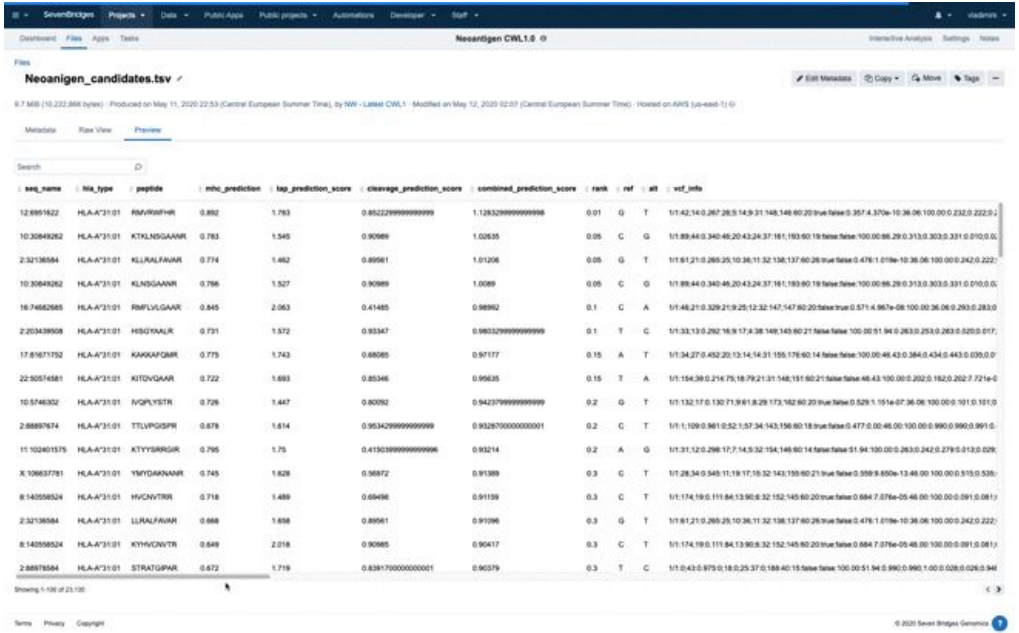
Neoantigen Discovery Workflow



130 bioinformatic tools, Execution time 7 to 14 hours

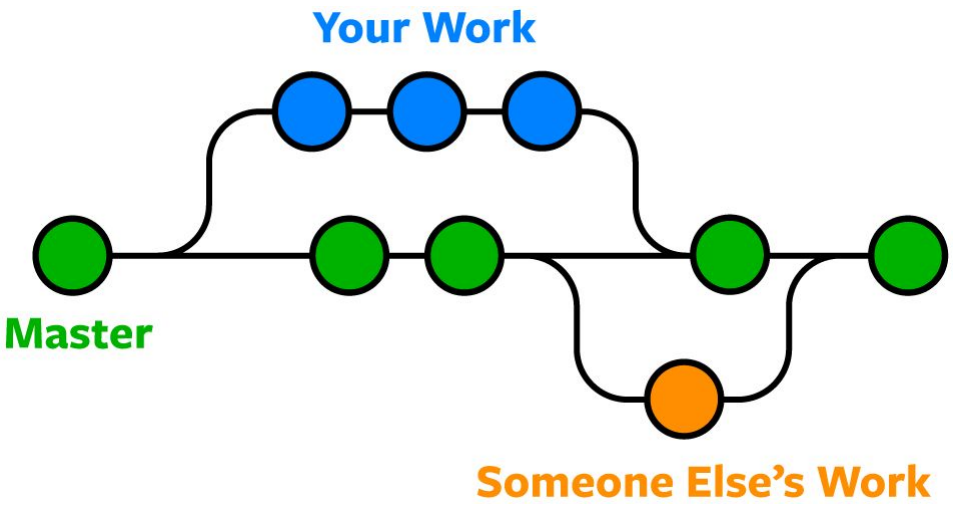
Scored Neoantigen Candidates

- Basic variant and neoantigen info
- MHC binding scores from set of tools
  - NetCTLPan
  - NetMHCPan
  - Pickpocket
  - MHCNuggets
- Agretopicity (difference between MHC binding mutated and reference peptide)
- Tumor and normal VAF
- Presence in wild type protein database



peptide	hla_type	variant id	vcf info	netctlpan cps	netmhcpan ic50	IEDB ic50	pickpocket ic50	mhcnuggets ic50	wild type protein	agretopicity	tumor vaf	normal vaf	expressed isoforms	rna tpm	rna variant called	rna AD	rna DP	rna GT	alt allele exp	ref allele exp
KTKLNSGAANR	HLA-A*31:01	22_44343533_A_C	1/1:89;44:0.3	1.02635	16.19	16.19	21.5	92.47	0	0.021	0.331	0	ENST00000538351	1.895323	TRUE	12	24	0/1	15	1

Why Git?



We use Git!

- Created by Linus Torvalds, creator of Linux, in 2005
- Came out of Linux development community
- Designed to do version control on Linux kernel
- Goals of Git:
  - Speed
  - Support for non-linear development (thousands of parallel branches)
  - Fully distributed
  - Able to handle large projects efficiently

(A "git" is a cranky old man. Linus meant himself.)

- Instructions to install Git: https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

Installing/learning Git!

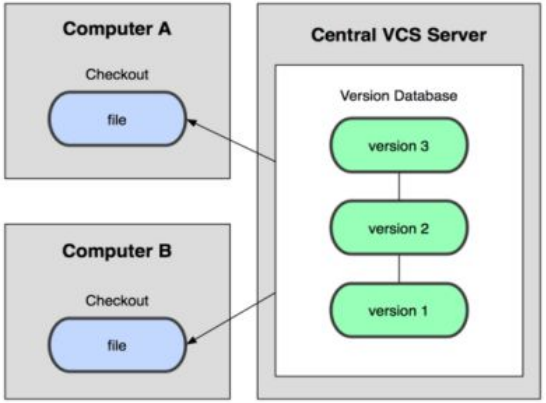
- Git website: http://git-scm.com/
- Free online book: http://git-scm.com/book
- Reference page for Git: http://gitref.org/index.html
- Git tutorial: http://schacon.github.com/git/gittutorial.html
- Git slides: https://courses.cs.washington.edu/courses/cse403/13au/lectures/git.ppt.pdf
- Git for Computer Scientists: http://eagain.net/articles/git-for-computer-scientists
- At command line: (where verb = config, add, commit, etc.)

```
git help verb
```

- Instructions to install Git: https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

Centralized Versioning Control System

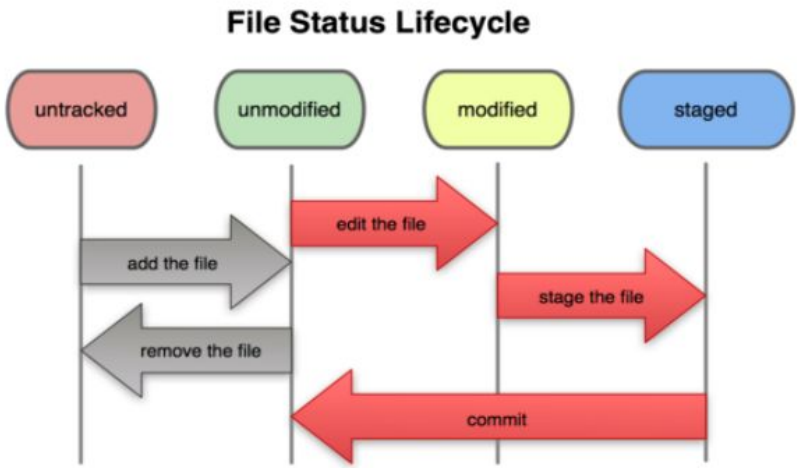
- A central server repository (repo) holds the "official copy" of the code
- The server maintains the sole version history of the repo
- You make "checkouts" of it to your local copy
- You make local modifications
- Your changes are not versioned
- When you're done, you "check in" back to the server
- your check in increments the repo's version



Basic Git flow

1. Modify files in your working directory
2. Stage files, adding snapshots of them to your staging area
3. Commit, which takes the files in the staging area
4. Store that snapshot permanently to your Git directory

```
git add file.py
git commit -m "Description of change."
git push origin master
```



Initial Git configuration

Set the name and email for Git to use when you commit:

- git config --global user.name "Bugs Bunny"
- git config --global user.email bugs@gmail.com

You can call git config –list to verify these are set.

Git commands

command	description
`git clone url [dir]`	copy a Git repository so you can add to it
`git add file`	adds file contents to the staging area
`git commit`	records a snapshot of the staging area
`git status`	view the status of your files in the working directory and staging area
`git diff`	shows diff of what is staged and what is modified but unstaged
`git help [command]`	get help info about a particular command
`git pull`	fetch from a remote repo and try to merge into the current branch
`git push`	push your new branches and data to a remote repository
`git checkout filename`	undoes your changes
Others: init, reset, branch, checkout, merge,log, tag	

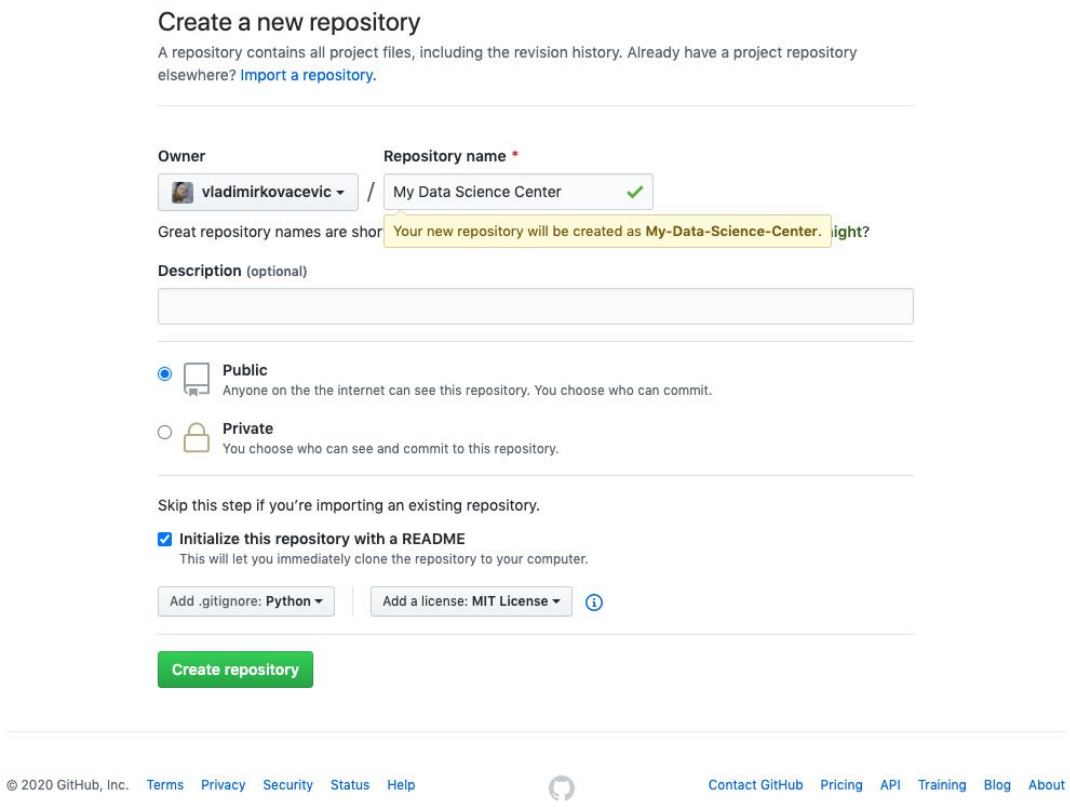
We use Github!

- GitHub.com is a site for online storage of Git repositories.
- You can create a remote repo there and push code to it.
- Many open source projects use it, such as the Linux kernel.
- You can get free space for open source projects, or you can pay for private projects.
- Free private repos for educational use: github.com/edu
- Question: Do I always have to use GitHub to use Git?
    - Answer: No! You can use Git locally for your own purposes.
    - Or you or someone else could set up a server to share files.
    - Or you could share a repo with users on the same file system, as long everyone has the needed file permissions).

Setup Github repo

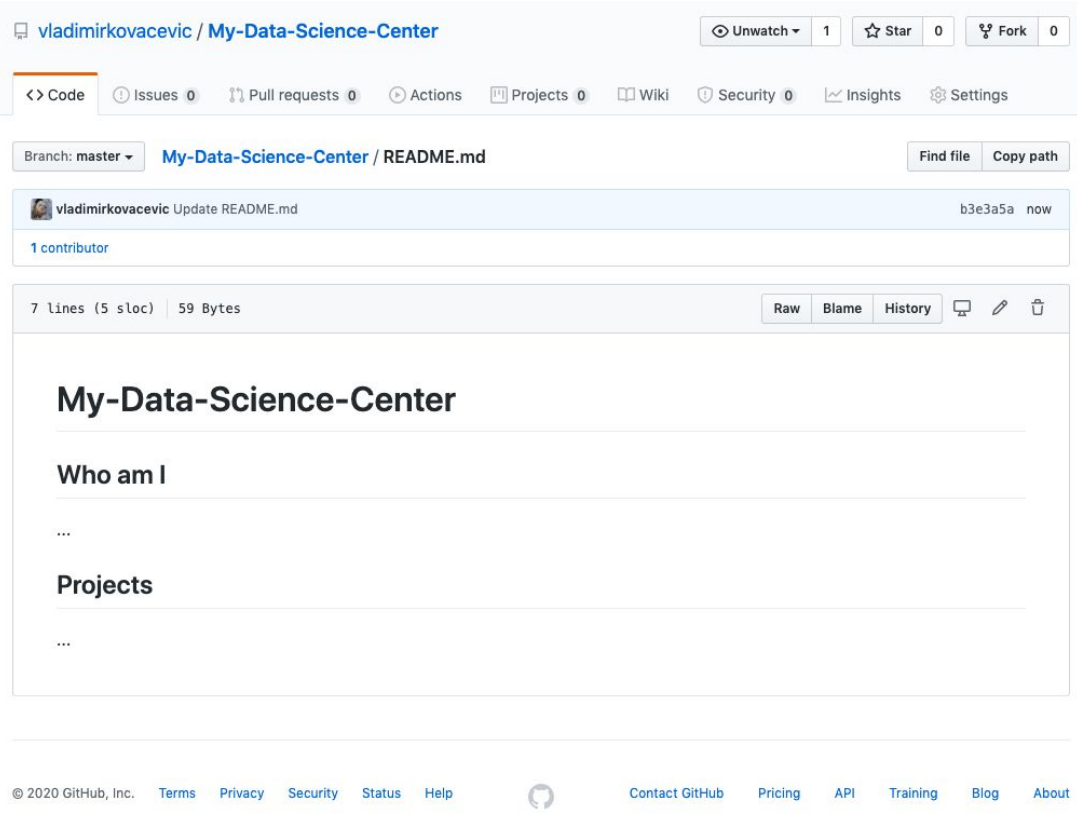
- Create account on www.github.com
- Set an image :)
- Create repository

  My Data Science Center
  - Initialize with README
  - .gitignore Python
  - MIT License



Setup Github repo

- Add short biography
- Projects will come on the way



Resources and additional reads

Presentation available at: github.com/vladimirkovacevic/gi-2026-etf

- A Computer Scientist's Guide to Cell Biology, A Travelogue from a Stranger in a Strange Land
- Genomics 101, Edition 2016
- Bioinformatics at COMAV - SNP Calling
- Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM
- High-Throughput Sequencing Technologies - Review paper
- Vince Buffalo: Bioinformatics Data Skills
- Dan Gusfield: Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Cambridge
- Pavel Pevzner, Neils Jones: An Introduction to Bioinformatics Algorithms (Computational Molecular Biology), MIT
- R. Durbin, S. Eddy, A. Krogh, G. Mitchinson: Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids, Cambridge University Press
- Veli Mäkinen, Djamal Belazzougui, Fabio Cunial, Alexandru I. Tomescu: Genome-Scale Algorithm Design: Biological Sequence Analysis in the Era of High-Throughput Sequencing, Cambridge University press
- Molekularna biologija 1; Dušanka Savić-Pavićević, Gordana Matić; NNK International, 2020
- RNA-seqlopedia; Cresko Lab, University of Oregon